

**FINAL PROJECT SUBMISSION REPORT OF
ELEVATE LABS CYBERSECURITY
INTERNSHIP**

NAME	Jadav Dinesh
Submitted to:	Elevate Labs
Name of the Academic Institute	Ganpat University

REPORT SUBMITTED TO



As part of the Cyber Security Internship, I have completed "Final Project : Personal Firewall using Python Report" by following all steps as instructed. Below is a detailed summary of each step followed during the task:

Final project: Personal Firewall using Python Report

Introduction

This project focuses on developing a lightweight personal firewall using Python. The firewall is designed to monitor network traffic, filter packets based on user-defined rules, and optionally interface with system-level firewalls such as iptables. It can also provide a GUI for real-time monitoring.

Abstract

The personal firewall is a Python-based application leveraging Scapy to sniff network packets. Users can define rules to block or allow specific IPs, ports, and protocols. Suspicious packets are logged for auditing purposes. The project demonstrates core concepts in network security and offers both CLI and GUI interfaces for flexibility.

Tools Used

- **Python 3.13.3**
- **Scapy (for packet sniffing)**
- **iptables (for system-level enforcement, Linux)**
- **Tkinter (for optional GUI)**

Steps Involved in Building the Project

- 1. Initialized packet sniffing using Scapy to capture live traffic.**
- 2. Designed a rule-matching system to filter packets by IP, port, and protocol.**
- 3. Implemented logging of blocked packets into a text file.**
- 4. Developed a Tkinter GUI to allow rule management and display live logs.**
- 5. Enabled optional integration with iptables for advanced blocking at the OS level**
- 6. Tested the firewall against various IP traffic scenarios in a controlled environment.**

Final project: Personal Firewall using Python Report

```
1 # SPDX-License-Identifier: GPL-2.0-only
2 # This file is part of Scapy
3 # See https://scapy.net/ for more information
4 # Copyright (C) Philippe Biondi <phil@secdev.org>
5
6 """
7 Aggregate top level objects from all Scapy modules.
8 """
9
10 from scapy.base_classes import *
11 from scapy.config import *
12 from scapy.dadict import *
13 from scapy.data import *
14 from scapy.error import *
15 from scapy.themes import *
16 from scapy.arch import *
17 from scapy.interfaces import *
18
19 from scapy.plist import *
20 from scapy.fields import *
21 from scapy.packet import *
22 from scapy.asn1fields import *
23 from scapy.asn1packet import *
24
25 from scapy.utils import *
26 from scapy.route import *
27 from scapy.sendrecv import *
28 from scapy.sessions import *
29 from scapy.supersocket import *
30 from scapy.volatile import *
31 from scapy.as_resolvers import *
```

```
32
33 from scapy.automaton import *
34 from scapy.autorun import *
35
36 from scapy.main import *
37 from scapy.consts import *
38 from scapy.compat import raw # noqa: F401
39
40 from scapy.layers.all import *
41
42 from scapy.asn1.asn1 import *
43 from scapy.asn1.ber import *
44 from scapy.asn1.mib import *
45
46 from scapy.pipetool import *
47 from scapy.scapypipes import *
48
49 if conf.ipv6_enabled: # noqa: F405
50     from scapy.utils6 import * # noqa: F401
51     from scapy.route6 import * # noqa: F401
52
53 from scapy.ansmachine import *
54
55
56 def IP():
57     return None
58
59
60 def TCP():
61     return None
62
63
64 def UDP():
65     return None
```

Final project: Personal Firewall using Python Report

```
BCI FIREWALL.PY  Version control
firewall.py x  all.py

1  import tkinter as tk
2  from tkinter import messagebox, scrolledtext
3  import threading
4  from scapy.all import sniff, IP, TCP, UDP
5  from datetime import datetime
6
7  # === Global Variables ===
8  firewall_rules = []
9  log_file = "firewall_log.txt"
10
11
12 # === Packet Rule Matching ===
13 def check_rules(packet): 1usage
14     if IP in packet:
15         ip_src = packet[IP].src
16         ip_dst = packet[IP].dst
17         proto = "TCP" if packet.haslayer(TCP) else "UDP" if packet.haslayer(UDP) else "OTHER"
18         port = packet[TCP].dport if packet.haslayer(TCP) else (packet[UDP].dport if packet.haslayer(UDP) else None)
19
20         for rule in firewall_rules:
21             if rule['ip'] and rule['ip'] not in (ip_src, ip_dst):
22                 continue
23             if rule['port'] and rule['port'] != port:
24                 continue
25             if rule['proto'] and rule['proto'] != proto:
26                 continue
27             return rule['action']
28     return "ALLOW"
29
30
31 # === Logging Function ===
32 def log_packet(packet, reason="BLOCKED"): 1usage
33     with open(log_file, "a") as f:
34         log = f"{datetime.now()} | {packet.summary()} | {reason}\n"
35         f.write(log)
36         gui_log(log)
37
38
39 # === GUI Log Viewer ===
```

```
BCI FIREWALL.PY  Version control
firewall.py x  all.py

39 # === GUI Log Viewer ===
40 def gui_log(msg): 2 usages
41     log_display.configure(state='normal')
42     log_display.insert(tk.END, msg)
43     log_display.see(tk.END)
44     log_display.configure(state='disabled')
45
46
47 # === Scapy Packet Sniffing Handler ===
48 def packet_handler(packet): 1usage
49     action = check_rules(packet)
50     if action == "BLOCK":
51         log_packet(packet)
52     else:
53         log_msg = f"{datetime.now()} | {packet.summary()} | ALLOWED\n"
54         gui_log(log_msg)
55
56
57 # === Threaded Sniff Function ===
58 def start_sniffing(): 1usage
59     sniff(filter="ip", prn=packet_handler, store=0)
60
61
62 # === Rule Addition ===
63 def add_rule_gui(): 1usage
64     ip = ip_entry.get().strip()
65     port = port_entry.get().strip()
66     proto = proto_entry.get().strip().upper()
67     action = action_var.get().upper()
68
69     try:
70         port = int(port) if port else None
71     except ValueError:
72         messagebox.showerror(title="Error", message="Port must be an integer!")
73         return
74
75     rule = {'ip': ip if ip else None, 'port': port, 'proto': proto if proto else None, 'action': action}
76     firewall_rules.append(rule)
77     rule_list.insert(0, f"{rule['ip']} | {rule['port']} | {rule['proto']} | {rule['action']}")
```

Final project: Personal Firewall using Python Report

```
BCI FIREWALL.PY Version control
firewall.py x all.py

...

60
61
62 # === Rule Addition ===
63 def add_rule_gui():
64     ip = ip_entry.get().strip()
65     port = port_entry.get().strip()
66     proto = proto_entry.get().strip().upper()
67     action = action_var.get().upper()
68
69     try:
70         port = int(port) if port else None
71     except ValueError:
72         messagebox.showerror("Error", "Port must be an integer!")
73         return
74
75     rule = {'ip': ip if ip else None, 'port': port, 'proto': proto if proto else None, 'action': action}
76     firewall_rules.append(rule)
77     rule_list.insert(tk.END, f"{action} - IP: {ip or 'ANY'}, Port: {port or 'ANY'}, Proto: {proto or 'ANY'}")
78
79
80 # === GUI Setup ===
81 app = tk.Tk()
82 app.title("Python Personal Firewall")
83 app.geometry("700x500")
84
85 # Rule Frame
86 tk.Label(app, text="IP:").grid(row=0, column=0)
87 tk.Label(app, text="Port:").grid(row=1, column=0)
88 tk.Label(app, text="Protocol:").grid(row=2, column=0)
89
90 ip_entry = tk.Entry(app)
91 port_entry = tk.Entry(app)
92 proto_entry = tk.Entry(app)
93
94 ip_entry.grid(row=0, column=1)
95 port_entry.grid(row=1, column=1)
96 proto_entry.grid(row=2, column=1)
97
98 action_var = tk.StringVar(value="BLOCK")
99
100 # GUI Setup (continued)
101 app = tk.Tk()
102 app.title("Python Personal Firewall")
103 app.geometry("700x500")
104
105 # Rule Frame
106 tk.Label(app, text="IP:").grid(row=0, column=0)
107 tk.Label(app, text="Port:").grid(row=1, column=0)
108 tk.Label(app, text="Protocol:").grid(row=2, column=0)
109
110 ip_entry = tk.Entry(app)
111 port_entry = tk.Entry(app)
112 proto_entry = tk.Entry(app)
113
114 ip_entry.grid(row=0, column=1)
115 port_entry.grid(row=1, column=1)
116 proto_entry.grid(row=2, column=1)
117
118 action_var = tk.StringVar(value="BLOCK")
119 tk.Radiobutton(app, text="Block", variable=action_var, value="BLOCK").grid(row=3, column=0)
120 tk.Radiobutton(app, text="Allow", variable=action_var, value="ALLOW").grid(row=3, column=1)
121 tk.Button(app, text="Add Rule", command=add_rule_gui).grid(row=4, column=0, columnspan=2)
122
123 # Rule Listbox
124 tk.Label(app, text="Active Rules:").grid(row=5, column=0, sticky="w")
125 rule_list = tk.Listbox(app, height=5, width=80)
126 rule_list.grid(row=6, column=0, columnspan=3)
127
128 # Log Display
129 tk.Label(app, text="Packet Logs:").grid(row=7, column=0, sticky="w")
130 log_display = scrolledtext.ScrolledText(app, width=85, height=10, state="disabled")
131 log_display.grid(row=8, column=0, columnspan=3)
132
133 # Start Sniffing in Background
134 sniffer_thread = threading.Thread(target=start_sniffing, daemon=True)
135 sniffer_thread.start()
```

Final project: Personal Firewall using Python Report

```
BCI FIREWALL.PY Version control
firewall.py x all.py
86 tk.Label(app, text="IP:").grid(row=0, column=0)
87 tk.Label(app, text="Port:").grid(row=1, column=0)
88 tk.Label(app, text="Protocol:").grid(row=2, column=0)
89
90 ip_entry = tk.Entry(app)
91 port_entry = tk.Entry(app)
92 proto_entry = tk.Entry(app)
93
94 ip_entry.grid(row=0, column=1)
95 port_entry.grid(row=1, column=1)
96 proto_entry.grid(row=2, column=1)
97
98 action_var = tk.StringVar(value="BLOCK")
99 tk.Radiobutton(app, text="Block", variable=action_var, value="BLOCK").grid(row=3, column=0)
100 tk.Radiobutton(app, text="Allow", variable=action_var, value="ALLOW").grid(row=3, column=1)
101
102 tk.Button(app, text="Add Rule", command=add_rule_gui).grid(row=4, column=0, columnspan=2)
103
104 # Rule Listbox
105 tk.Label(app, text="Active Rules:").grid(row=5, column=0, sticky="w")
106 rule_list = tk.Listbox(app, height=5, width=80)
107 rule_list.grid(row=6, column=0, columnspan=3)
108
109 # Log Display
110 tk.Label(app, text="Packet Logs:").grid(row=7, column=0, sticky="w")
111 log_display = scrolledtext.ScrolledText(app, width=85, height=18, state='disabled')
112 log_display.grid(row=8, column=0, columnspan=3)
113
114 # Start Sniffing in Background
115 sniffer_thread = threading.Thread(target=start_sniffing, daemon=True)
116 sniffer_thread.start()
117
118 app.mainloop()
119
```

FINAL RESULT

Python Personal Firewall

IP: 192.172.10.0

Port: 22

Protocol: TCP

☒ Block ☐ Allow

Add Rule

Active Rules:

BLOCK - IP: 192.172.10.0, Port: ANY, Proto: ANY
BLOCK - IP: 192.172.10.0, Port: 22, Proto: TCP

Packet Logs:

D

2025-06-21 17:27:59.445462 | Ether / IP / UDP / DNS Ans b'ocsp.entrust.net.edgekey.net.' | ALLOWED

2025-06-21 17:27:59.448928 | Ether / IP / UDP / DNS Ans b'ocsp.entrust.net.edgekey.net.' | ALLOWED

2025-06-21 17:28:04.473420 | Ether / IP / TCP 172.20.10.3:49414 > 4.213.25.241:https A / Raw | ALLOWED

2025-06-21 17:28:04.579818 | Ether / IP / TCP 4.213.25.241:https > 172.20.10.3:49414 A | ALLOWED

Final project: Personal Firewall using Python Report

Source code :-

1. <https://github.com/gantmama/project-firewall/pull/1/commits/08f2ac904516c7e80fcc630edaecc033fd4b9e33>
2. <http://github.com/gantmama/project-firewall/blob/projects/scapy%5Call.py>

Conclusion

This project provided hands-on experience in Python programming, packet analysis, and network security. The personal firewall can be a practical tool for basic traffic monitoring and rule-based filtering. It serves as an excellent foundation for further exploration into advanced firewall technologies and intrusion detection systems.